

边缘计算环境下基于 Flink 的实时数据流处理与动态资源调度优化

王五¹ 赵六²

¹ 计算机学院, 深圳 518172

² 哈尔滨工业大学(深圳) 计算机科学与技术学院, 深圳 518055

摘要: 随着物联网设备的快速普及和海量实时数据的产生, 传统云计算架构在处理边缘侧实时数据时面临高延迟、带宽瓶颈和隐私保护不足等挑战。本文针对边缘计算环境下的实时数据流处理问题, 提出了一种基于 Apache Flink 的分布式流处理框架, 并设计了动态资源调度优化策略。实验结果表明, 所提框架在毫秒级延迟约束下, 吞吐量较基准方法提升了 32%, 资源利用率提高了 28%。

关键词: 边缘计算; 实时流处理; Apache Flink; 动态资源调度

引言

边缘计算作为一种新型计算范式, 通过将计算能力从云端下沉到网络边缘, 为终端设备提供就近服务 [1]。随着 5G 技术的商用化推进, 边缘计算在智能制造、自动驾驶、智慧医疗等领域的应用前景愈发广阔。

实时数据流处理是边缘计算的核心需求之一。与传统批处理不同, 流处理要求系统在毫秒级延迟内完成数据采集、过滤、聚合和响应。Apache Flink [2] 作为一个真正的流处理引擎, 以其精确一次 (Exactly-once) 语义和低延迟特性, 成为边缘计算场景下的首选流处理框架。

系统架构

云-边-端三层架构

本文采用“云-边-端”三层协作架构。云端负责模型训练、全局调度和历史数据存储; 边缘层部署 Flink 集群, 负责实时流处理和本地决策; 终端层负责数据采集和指令执行。

Flink 流处理引擎

在边缘节点上, 我们部署了轻量化的 Flink 实例, 配置了基于 RocksDB 的状态后端以支持大状态管理, 并启用了检查点 (Checkpoint) 机制保障容错性。

(状态后端配置) RocksDB 状态后端能将状态数据持久化到本地磁盘, 适合大状态场景。在边缘节点内存有限的条件下, 这一特性尤为重要。

(检查点优化) 采用增量检查点策略, 仅保存自上次检查点以来的状态变更, 大幅减少了检查点开销。

动态资源调度策略

问题建模

边缘节点的计算资源有限且动态变化, 因此资源调度问题可以建模为约束优化问题。设边缘节点集合为 $\mathcal{N} = \{n_1, n_2, \dots, n_k\}$, 任务集合为 $\mathcal{T} = \{t_1, t_2, \dots, t_m\}$ 。

目标函数为最小化加权延迟与资源成本的

算法 1: 在线动态资源调度算法

输入: 任务队列 $\mathcal{Q}$, 节点列表 $\mathcal{N}$, SLA
约束 $L_{\max}$
输出: 任务-节点映射 $\mathcal{M}$

```

1 foreach  $t \in \mathcal{Q}$  do
2    $\mathcal{C} \leftarrow \{n \in \mathcal{N} \mid \text{满足资源约束}\};$ 
3   if  $\mathcal{C} = \emptyset$  then
4     将  $t$  加入等待队列;
5     continue;
6    $n^* \leftarrow \arg \min_{n \in \mathcal{C}} \text{Score}(t, n);$ 
7   if  $\text{Score}(t, n^*) \leq L_{\max}$  then
8      $\mathcal{M}[t] \leftarrow n^*;$ 
9     更新  $n^*$  的资源状态;
10  else
11    将  $t$  升级到云端处理;
12 return  $\mathcal{M}$ 

```

组合:

$$\min \sum_{i=1}^m \sum_{j=1}^k (w_1 \cdot x_{ij} \cdot L(t_i, n_j) + w_2 \cdot x_{ij} \cdot C(t_i, n_j)) \quad (.1)$$

约束条件包括:

- 每个任务必须分配到恰好一个节点:
 $\sum_j x_{ij} = 1, \forall i$
- 节点资源上限约束: $\sum_i x_{ij} \cdot r(t_i) \leq R(n_j), \forall j$
- 延迟 SLA 约束: $L(t_i, n_j) \leq L_{\max}, \forall (i, j) :$
 $x_{ij} = 1$

算法设计

采用改进的启发式贪心算法进行在线调度, 如算法1所示。

实验评估**实验环境**

实验使用 3 台边缘服务器 (Intel Xeon E-2278G, 32GB RAM) 和 10 台模拟终端设备。Flink 集群配置为 3 个 TaskManager, 每个分配 8GB 堆内存。

框架	低负载	高负载
Apache Storm	12,500	38,200
Spark Streaming	9,800	45,600
本文方法	18,200	60,800

表 1: 流处理框架吞吐量对比 (条/秒)

吞吐量对比

表1展示了不同框架在相同工作负载下的吞吐量对比。

延迟分析

在低负载 (1000 条/秒) 条件下, 本文方法的 P99 延迟为 8.2ms, 显著优于 Storm 的 15.4ms 和 Spark Streaming 的 120ms (微批次导致)。在高负载 (5000 条/秒) 条件下, 本文方法通过动态调度策略仍将 P99 延迟控制在 23.5ms 以内。

资源利用率

动态调度策略使 CPU 利用率从静态分配方案的 62% 提升至 78%, 内存利用率从 55% 提升至 72%。

相关工作

边缘计算的概念最早由 Shi 等人 [1] 系统阐述。在流处理方面, Carbone 等人 [2] 提出的 Apache Flink 以其真正的流处理语义成为业界标准。近年来, 研究者们提出了多种边缘-云端协同的流处理方案。

与现有工作相比, 本文的主要区别在于: (1) 针对边缘资源受限场景设计了轻量化 Flink 配置; (2) 提出了在线动态资源调度算法, 而非静态分配; (3) 在真实边缘服务器环境下进行了充分验证。

结论与展望

本文提出了一种基于 Apache Flink 的边缘实时数据流处理框架, 并设计了动态资源调度优化策略。实验结果表明, 所提方法在吞吐量和资源利用率方面均有显著提升。

未来的研究方向包括: 探索基于强化学习



图 1: 云-边-端三层系统总体架构（跨栏排版）

的智能调度策略、支持异构硬件（GPU/FPGA）的流处理加速、以及在更大规模边缘集群上进行验证。

参考文献

[1] Shi W, Cao J, Zhang Q, et al. Edge computing: Vision and challenges[J]. IEEE Internet of Things Journal, 2016, 3(5): 637-646.

[2] Carbone P, Katsifodimos A, Ewen S, et al. Apache Flink: Stream and batch processing in

a single engine[J]. Bulletin of the IEEE Computer Society Technical Committee on Data Engineering, 2015, 38(4): 28-38.

[3] Toshniwal A, Taneja S, Shukla A, et al. Storm@Twitter[C]. SIGMOD, 2014: 147-156.

[4] Zaharia M, Xin R S, Wendell P, et al. Apache Spark: A unified engine for big data processing[J]. Communications of the ACM, 2016, 59(11): 56-65.